devops engineer - detailed guide

DevOps Engineer — Detailed Guide

For someone doing the role. Concepts tied to the actual files in this repository. Read `easy-guide.md` first if any term here is unfamiliar.

Working assets: `infra/` , `docker-compose.yml` , `docker-compose.inference.yml` , `backend/Dockerfile` , `tasks.py` , `Makefile` , `.github/workflows/ci.yml`

This guide explains *why* the infra is shaped this way. It intentionally does not restate command-by-command operating procedure — that lives in `infra/docs/` , which is operational documentation, not study material. Follow the links; don’t duplicate them from memory.

---

1. Scope: what “the deployable unit” actually is

```
backend/                    the FastAPI service
frontend/                   the React app + nginx
backend/data/               the 130 MB generated data layer (NOT in git)
docker-compose.yml          the stack definition
infra/compose/*.prod.yml    production hardening overlay
```

`research/` — the model training tree — is **not part of a deployment**. This is the load-bearing architectural decision in this whole project: the API was verified to run correctly against a completely empty `research/` directory. Every route responds — the frozen 28-day forecast, hierarchy, accuracy, insights, `/docs`, every `/genai` route — and `/inference/status` correctly reports itself unavailable with a reason, rather than crashing.

The one exception, live model re-verification (`--inference`), is opt-in specifically so the *default* deployment carries zero dependency on the research tree. Understanding this split — product-data-layer vs. research-tree — is the key to understanding almost every other decision in `infra/`.

Full reasoning: `infra/docs/deployment-guide.md` §1-2.

---

2. The data-layer contract (`backend/app/config.py`)

`Settings.required_artefacts` (`config.py:175-180`) names the three files the API needs at `NPN_DATA_DIR`:

```
product.duckdb      20 MB
history.parquet     32 MB
backtest.parquet    78 MB
```

These are **generated**, by `backend/scripts/build_product_db.py`, from frozen research artefacts. They are gitignored and always rebuildable — so they never need backing up. They must simply *exist* before the API can report itself ready.

This single fact drives four different pieces of infra you'll touch:

Consequence	Where it's enforced
A pre-deploy check must confirm the files exist and are the right size	`infra/scripts/preflight.py` → `check_data_layer()`
A readiness probe must prove the *data*, not just the *process*, is up	`/api/v1/ready` vs. `/api/v1/health` — see §4
A post-deploy check must prove the *served numbers* are the *validated* ones	`infra/scripts/smoke_test.py` — the `3331.3681` canary
The compose file must mount it, read-only	`docker-compose.yml` `volumes:` on the `api` service

Four deployment strategies for getting this file onto a target host or cluster — bind mount (current, simplest), a baked "data image", a Kubernetes init container, or fetching from object storage — are compared in `deployment-guide.md` §2. Bind mount is right for a single host; a baked data image is recommended if you ever move this to a cluster, because the files are immutable and version- independent.

---

3. The two-target Dockerfile (`backend/Dockerfile`)

One Dockerfile, two build targets, sharing a base and a dependency layer structure:

```
base ──► deps-api  ──► api    (default; no ML libs, no research mount needed)
     └─► deps-full ──► full   (+ lightgbm/numpy/pandas/pyarrow; opt-in)
```

Both targets share the pattern of copying `requirements.txt` **before** application code, so a code-only change doesn't invalidate the dependency install layer — a standard multi-stage caching technique, but worth noticing because it's *why* the layer ordering is what it is.

Non-root from the start. `useradd --uid 10001 ... app` runs before anything else in `base`, and both targets end with `USER app`. The API only ever *reads* its mounted data, so it never needs write access to anything — running as root would be a capability with no matching need. CI's `images` job asserts this directly: `docker run --entrypoint id` must report uid **10001**.

Healthcheck uses `/ready`, not `/health` — deliberately, matching §4.

The `full` target's import-time `mkdir` problem is worth understanding even if you never touch it, because it's a real class of bug: `pipeline/config.py` calls `mkdir(parents=True, exist_ok=True)` on seven directories **at import time**. With `NPN_PROJECT_ROOT=/research`, three of those don't otherwise exist, and `/research` itself would be root-owned in a fresh container — so importing the pipeline as uid 10001 fails with `PermissionError` on first use. The Dockerfile pre-creates and `chown`s all ten resulting paths (lines ~99-107) specifically to avoid this. If you ever see this `PermissionError` in the wild, the fix is to restore that block — **not** to run the container as root. See `troubleshooting.md` #10.

Build context is the repository root, not `backend/` — because the `full` target needs `COPY research/pipeline`. That single fact is *why* `.dockerignore` lives at the repo root and *why* it has to enumerate every top-level directory (§6).

---

4. Liveness vs. readiness — where it's actually wired

Probe	Endpoint	Proves	Set as
Liveness	`/api/v1/health`	the process is alive	`livenessProbe`
Readiness	`/api/v1/ready`	the data layer is **queryable**	`readinessProbe`

Both `docker-compose.yml`'s `healthcheck:` block and the Dockerfile's own `HEALTHCHECK` instruction point at `/ready`, and `frontend` depends on `api`'s `service_healthy` condition. That dependency chain is *exactly* why a missing data layer looks like a hang rather than a clear error: the API container never reaches `(healthy)`, so the frontend container never starts at all, and `docker compose up` just appears to sit there.

`preflight` now runs automatically before `docker-up` specifically to catch this before it ever reaches that state.

If you write Kubernetes manifests (none exist yet — see §9), the mapping is direct: `readinessProbe` → `/ready`, `livenessProbe` → `/health`, with `~15s` initial delay (DuckDB has to open at startup) and an extra ~10s if you're running the `full` image, which imports a much heavier dependency stack.

---

5. `docker-compose.yml` and the production overlay

The base compose file defines two services, `api` and `frontend`, on a shared user-defined network — which matters because nginx's `envsubst`- rendered `proxy_pass` resolves the upstream hostname **at startup** and refuses to boot if it can't (`troubleshooting.md` #6). This never bites under Compose, only if you run containers by hand on the default bridge network.

Key design points baked into the base file:

- `GEMINI_API_KEY: ${GEMINI_API_KEY:-}` — substitution syntax, never a literal. The default-empty form means "unset" is a fully supported state, not an error.
- The data volume is mounted `:ro` — the API process never has write access to its own data layer, structurally, not by convention.
- `NPN_CORS_ORIGINS` is set to dev localhost origins by default, because in the normal topology **CORS is never actually exercised** — the browser only ever talks to the frontend's nginx, which proxies `/api/` internally on the same origin. CORS only matters for split-origin deployments or `npm run dev`.

`infra/compose/docker-compose.prod.yml` layers on top (`docker-up --prod`): API off the host network entirely (reachable only through nginx), read-only root filesystems, all Linux capabilities dropped, and rotating logs (10 MB × 5 files per service). **If a container that works without `--prod` fails with it, the near-universal cause is a missing writable path** — `read_only: true` doesn't allow a process to write anywhere that isn't explicitly given a `tmpfs` entry. The fix is always to add the *specific* path to `tmpfs`, never to drop `read_only`. nginx already has the three it needs (`/var/cache/nginx`, `/var/run`, `/tmp`); it also needs `CHOWN`, `SETGID`, `SETUID` capabilities restored, because its own entrypoint drops privileges itself — removing those three specific capabilities after `cap_drop: ALL` stops the container from booting at all.

`docker-compose.inference.yml` is the third, separate overlay: switches the API's build target from `api` to `full` and adds the two research-tree read- only mounts. Applied via `docker-up --inference`.

---

6. `preflight.py` — the pre-deploy gate

Six check groups, run from `infra/scripts/preflight.py`, each in its own section of the file:

Function	Checks	Needs Docker?
`check_data_layer()`	the three product files exist, with plausible sizes	No
`check_compose()`	both compose files parse; every service still has a healthcheck, restart policy, resource limits	No
`check_build_context()`	`.dockerignore` covers **every top-level directory**; total context size is small	No
`check_secrets()`	no literal API key anywhere in tracked files, including git history	No
`check_dockerfiles()`	pinned base image, present `HEALTHCHECK`, `USER` set **after** `CMD` never happens	No
`check_toolchain()`	Python/Node/Docker versions present and adequate	No

Nothing here needs Docker to be installed — that's a deliberate property, so preflight runs identically on a laptop and on a CI runner with no daemon available (`--skip-data` is what CI passes, acknowledging a runner can never have the data layer).

`check_build_context()` is the one worth understanding in depth, because it caught a real shipped bug (§8, incident 1). `.dockerignore` here is written as an **allow-list, enumerated per top-level directory** — not a deny-list. The file's own header explains why: a deny-list silently starts shipping anything added *later*, because nothing has to opt it out. But the allow-list form has a matching hole from the other direction — **a brand-new top-level directory is covered by nothing**, so it gets shipped by default until someone adds a rule for it. `preflight` now hard-**fails** (not warns) on any top-level directory with no matching rule, which converts "someone notices the build is slow" into "the very next preflight run catches it."

---

7. `smoke_test.py` — post-deploy verification

Stdlib-only, so it runs anywhere with a Python interpreter — a CI runner, a bastion host, or inside a container — with no dependency install needed. Three phases:

- `check_api()` — hits the running API's own endpoints, and asserts the frozen-forecast canary: `CA_3 / FOODS_3_090` must total **exactly 3331.3681** over 28 days. This is the check that distinguishes "responding" from "correct" — a stack can answer `200` on every route while quietly serving a stale or partially-built `product.duckdb`.
- `check_web()` — only runs when `--skip-web` isn't passed; verifies the frontend is serving, including that a client-side route survives a direct hit (the SPA-fallback / deep-link problem in

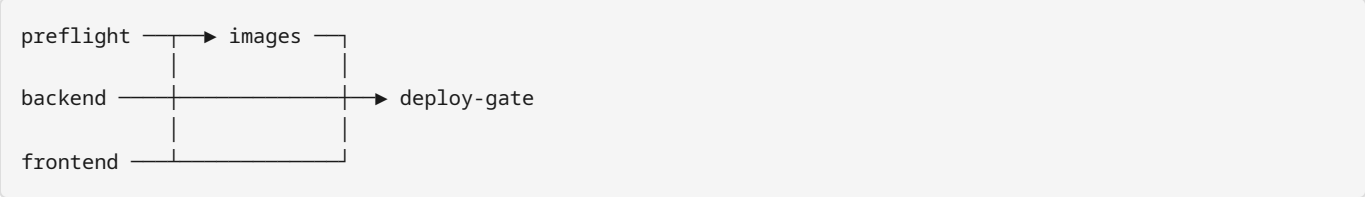
`troubleshooting.md` #7).
- `check_secrets()` — scans response bodies collected during the run for anything key-shaped, so a live check reconfirms what `preflight` already asserted statically.

`wait_for()` retries with a delay before failing, because DuckDB genuinely takes a moment to open at container startup — a smoke test with no retry budget would be a source of false failures, not a useful signal.

This is the script CI structurally cannot run for you, because a CI runner has no data layer to serve real numbers with (§8). Running it — by hand, against a real host — is the one verification step that has no substitute.

---

8. CI (`ci.yml`) — what it proves, and the boundary it can't cross



The whole pipeline is shaped by one constraint, stated explicitly at the top of `ci-cd.md`: **the 130 MB data layer, and the frozen model binaries it's generated from, are not in git.** A CI runner therefore can never reach `ready: true`, and any pipeline pretending otherwise would be either permanently red or quietly meaningless. So this pipeline is explicit about exactly what it can and can't prove:

Verified in CI	Verified only on a real host
compose validity, build-context hygiene, secret hygiene	the frozen-forecast canary (`3331.3681`)
frontend tests, typecheck, production build	data-dependent backend tests
no key in the built JS bundle	the API actually reaching `ready: true`
both image targets build; containers boot as uid 10001	end-to-end serving correctness
the API **degrades correctly** with no data mount at all	

That last row — "degrades correctly with no data" — is the `images` job's most important assertion: with the data mount absent, `/health` must still return 200 and `/ready` must return `false`, **with a structured reason**, never a crash or a hang. A regression there is exactly the failure mode that turns a missing mount into an unexplained production incident.

`deploy-gate` runs with `if: always()` and prints an explicit **"not proven by this run"** section — the host-side half of the release checklist — so a green tick is never mistaken for more assurance than CI can actually give. **There is deliberately no CD job.** No deployment target, cloud account, or registry exists for this project yet; the shape to add one when there is a target is documented in `ci-cd.md`, tag-by-`github.sha`, never `latest`.

9. `tasks.py` — the single command surface

`tasks.py` (not the `Makefile`) is the real entry point, because `make` wasn't installed on the Windows machine this project is demonstrated on. The `Makefile` is now a thin wrapper that just calls `tasks.py` — this was **not always true**: every target referenced folders renamed three commits earlier, so `make api`, `make test`, `make build-db` were all silently broken for a stretch, and nobody noticed because nobody had `make` to run them with. The fix wasn't just correcting the paths — it was removing the duplication that let the drift happen at all, by making the Makefile call `tasks.py` instead of repeating its logic.

The command surface, grouped by what it operates on:

Group	Commands
Data layer	`build-db`, `clean-db`
Run on host	`api`, `stop-api`, `ui`, `ui-install`, `ui-build`
Test	`test`, `ui-test`, `genai-check`, `verify-integrity`, `verify-all`
Containers	`docker-config`, `docker-build`, `docker-up` (`--prod`, `--inference`), `docker-down`, `docker-logs`, `docker-ps`
Gates	`preflight` (`--skip-data`), `smoke` (`--prod`, `--skip-web`, `--api`)
Misc	`openapi` (regenerate the committed schema)

`docker-up` runs `preflight` first, automatically — the gate isn't something you have to remember to run separately before starting the stack.

---

10. Observability (`backend/app/main.py`)

There's no metrics backend or log-aggregation stack in this project (that gap is named explicitly in §12). What exists instead is a small set of disciplined primitives, all inside `main.py`, that make the API's behaviour inspectable without any external tooling — worth knowing in detail because they're what you'll actually reach for while operating this system, and they're the foundation you'd build a real observability stack on top of rather than replace.

Structured logging

```
logging.basicConfig(
    level=getattr(logging, settings.log_level.upper(), logging.INFO),
```

```
    format='{"time":"%(asctime)s","level":"%(levelname)s",'
           '"logger":"%(name)s","msg":"%(message)s"}',
)
```

Every log line is a single-line JSON object, not free-text prose — a one-line change with real consequences: it means `docker compose logs api | grep -iE 'error|warn'` composes cleanly with `jq`, and it means a future log shipper (Fluent Bit, Vector, a cloud provider's own agent) can parse every line without a fragile regex. `NPN_LOG_LEVEL` gates verbosity; `DEBUG` is for active diagnosis only, per `environment-reference.md`.

Request correlation (`request_context` middleware, `main.py:121-137`)

```
rid = request.headers.get("X-Request-ID") or uuid.uuid4().hex[:12]
...
response.headers["X-Request-ID"] = rid
response.headers["X-Response-Time-ms"] = f"{ms:.1f}"
if ms > settings.slow_request_ms:
    log.warning("slow request rid=%s %s %.0fms", rid, request.url.path, ms)
```

Every response carries an `X-Request-ID` — reused from the caller if supplied, generated otherwise — and every server-side log line for that request includes the same `rid=`. This is what turns "a user says something failed" into "search the logs for this exact ID" instead of correlating by timestamp and hoping nothing else happened in the same second. It costs one extra header and one `uuid4()` call per request — cheap enough to leave on unconditionally, including in production.

`slow_request_ms` (default `1000`) means slow requests self-report — you don't need a profiler running to notice a regression, you need to grep logs for `"slow request"`.

Error responses carry the same correlation id

Every exception handler (`api_error_handler`, `validation_error_handler`, `unhandled_handler`, `main.py:148-183`) attaches `request.state.request_id` to the JSON error body it returns, and the catch-all handler (`unhandled_handler`) logs the **full traceback** server-side via `log.exception()` while returning the client only a generic message plus that same id:

```
log.exception("unhandled rid=%s path=%s", ..., request.url.path)
return _problem(request, 500, "internal_error",
                "an unexpected error occurred; see server logs",
                error_type=type(exc).__name__)
```

This is the same discipline the GenAI layer applies to provider exceptions (never forward raw exception text to a caller — see the GenAI detailed guide, §9) applied at the whole-API level: a caller gets an id to report, an operator gets the full detail in the log, and the two are joined by that id rather than by leaking the detail to the caller.

Readiness as a three-state signal, not a boolean (`routers/health.py`)

`/api/v1/ready` returns `ready` **and** `degraded` as separate booleans:

```
core_ok = bool(detail.get("tables"))
sidecars_ok = detail.get("history_queryable") and detail.get("backtest_queryable")
return {"ready": bool(core_ok), "degraded": bool(core_ok and not sidecars_ok), ...}
```

If the core forecast table is queryable but a sidecar (history or backtest) isn't, the API reports `ready: true, degraded: true` rather than collapsing that into a single healthy/unhealthy boolean. An orchestrator should treat `ready: false` as unhealthy (stop routing traffic) and `degraded: true` as a warning worth alerting on but not worth pulling the instance out of rotation for. `/ready` also returns `cache: cache_stats()` inline — cache hit/size information alongside the readiness signal, so a quick `curl` gives you both in one call rather than needing a separate endpoint for cache introspection.

What's genuinely absent, and the shape of closing it

No `/metrics` endpoint (nothing Prometheus-scrapeable), and nothing ships logs off-box — `docker compose logs` on the host is the entire log story today, rotated only under `--prod` (10 MB × 5 files/service; **unbounded** otherwise, which is the main operational reason to always run `--prod` for anything left running longer than a dev session). This is a named, deliberate gap for a single-host demo, not an oversight — see §12. If you're asked to close it: add a `/metrics` endpoint **alongside**, not instead of, `/health` and `/ready` — they answer different questions (is it alive / is it ready to serve / what are its aggregate numbers over time) and collapsing them loses information the way collapsing `ready` + `degraded` into one boolean would. Keep emitting the same structured JSON to stdout and point a log shipper at it, rather than inventing a second logging path.

---

11. Real incidents, and what they teach

Two are worth knowing in detail because each is a *pattern*, not a one-off:

The build-context leak (`.dockerignore`, allow-list hole)

`study-materials/` was added to the repo *after* `.dockerignore` was written. Because the ignore file is an allow-list keyed per top-level directory, the new directory matched no rule and all its PDFs were uploaded to the Docker daemon on every single build — measured at 0.70 MB → after the fix, 0.38 MB. The fix itself is one line; **the durable fix is that `preflight` now hard-fails on any top-level directory with no matching rule**, so the *next* new top-level directory is caught automatically rather than found by someone noticing a slow build. If you add a new top-level directory to this repo, adding a `.dockerignore` rule for it is not optional — `preflight` will block you on it, by design.

The `Makefile` silently broken for three commits

Covered in §9. The general lesson: **a tool nobody has installed cannot fail loudly.** If your team doesn't uniformly have a given tool, either don't depend on it for anything that matters, or make it a thin, low-surface-area wrapper over something that *is* uniformly available — which is exactly the fix that was applied here.

A structural rename bug is also worth knowing about even though it isn't an "incident" in the same sense: when `backend/` moved up one directory level in a repo reorganisation, every `Path(__file__).resolve().parents[N]` walk to the repository root that assumed the old depth was silently off by one. `.dockerignore`, being keyed to old directory names, had to be **rewritten**, not substituted — a plain find-and-replace would have left it referencing directories that no longer existed while covering nothing that did.

---

12. Known, deliberate gaps — and why they weren't built

Worth understanding *before* you're tempted to "fix" one of these, because each was considered and rejected for a specific reason, not overlooked:

Gap	Why it wasn't built
Kubernetes manifests	Untested manifests are worse than none — they look like a supported path that isn't. The mapping to write them from is documented, deliberately, instead.
A CD job	No deployment target, cloud account, or registry exists yet. A deploy job pointing at nothing looks like a capability that doesn't exist.
TLS, authentication, rate limiting	All three belong at the network edge, not inside these containers — and none of the three is optional if this is ever made public, particularly because `/genai/ask` costs money per call and `/inference/verify` costs ~1 GB and ~45s per call.
Metrics/log aggregation (see §10)	A real gap for a long-running deployment; a non-issue for a demo. Health, readiness, and structured logs are what an orchestrator actually needs day one.
In-container data bootstrap	Considered and rejected: it couldn't be tested without Docker installed, and an untested bootstrap path is worse than a documented one-line prerequisite (`build-db`).
More than one worker/replica in-process	`--workers 2` roughly doubles memory for zero throughput gain on the read path, because DuckDB opens read-only per process. Scale with **replicas** behind a load balancer instead — and note that inference jobs are held in-process, so they don't survive load balancing without sticky sessions.

13. Where to go next

- `infra/docs/environment-reference.md` — every environment variable, source-of-truth is `backend/app/config.py`.
- `infra/docs/deployment-guide.md` — the decisions you have to make deploying somewhere new (§2's four data- layer strategies, §3's per-platform notes for Cloud Run/ECS/K8s).
- `infra/docs/runbook.md` — day-to-day operation: the release checklist, rollback, key rotation, incident response.
- `infra/docs/troubleshooting.md` — the full symptom → cause → fix reference; skim it once so symptoms are recognisable later.
- `infra/docs/handover.md` — the honest state of this work: what's verified, what isn't (no image has ever actually been built — Docker isn't installed on the machine this was built on), and the exact first sequence of commands to run once it is.
- `docs/08_DEPLOYMENT/DOCKER_IMPLEMENTATION_REPORT.md` — background on how the containers were originally designed.